THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L07 Ensemble Method

Dr. Weikai Yang

Data Science and Analytics Thrust
& Computational Media and Arts Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

March 16, 2026

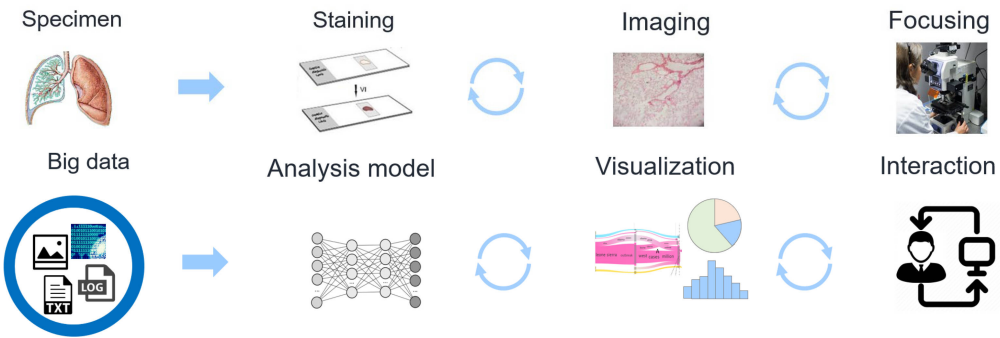
Course feedback

♡ Please scan the QR code and share your feedback.
No worries as it is anonymous. We sincerely appreciated your help.



About Me

- BS and PhD from Tsinghua University
- Research Interest: Visual Analytics + Data Centric ML
  - Help users better understand data and model



Syllabus

Lec #	Topic	Lecturer
1	Introduction + course Info	Zixin
2-4	Supervised learning: regression and classification	Zixin
5	Feature engineering + Model evaluation and choice	Zixin
6	Model evaluation and choice + Feature selection	Zixin
7	**Ensemble learning (supervised learning)**	Weikai
	Midterm (28 March, 1:30 - 3:00 PM): mark it on calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	Dimension reduction	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	

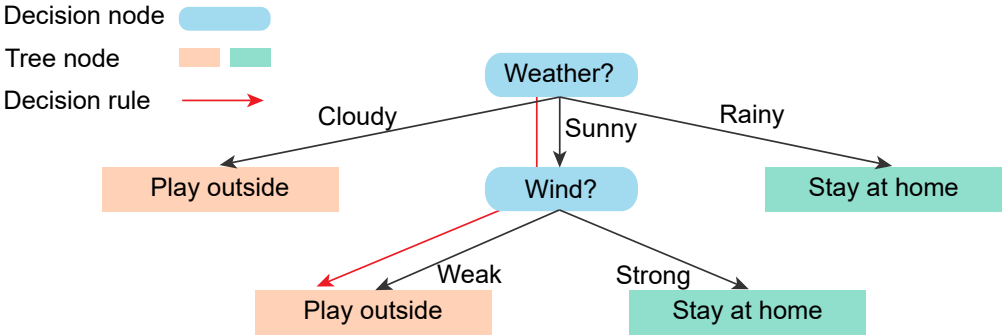


Outline

1. Recap: decision tree
2. Ensemble learning
3. Bagging (Bootstrap AGGregatING)
4. Random forest
5. Boosting
   - Gradient boosting
   - AdaBoost (Adaptive Boosting)



Recap of decision tree



What is a decision tree?

- Simple, tree-like model
- Make decisions by asking a series of questions (usually yes/no).
- Used for
  - *classification* (e.g., spam or not spam)
  - *regression* (e.g., predicting house prices)
- Think of it as a flowchart for decision-making!



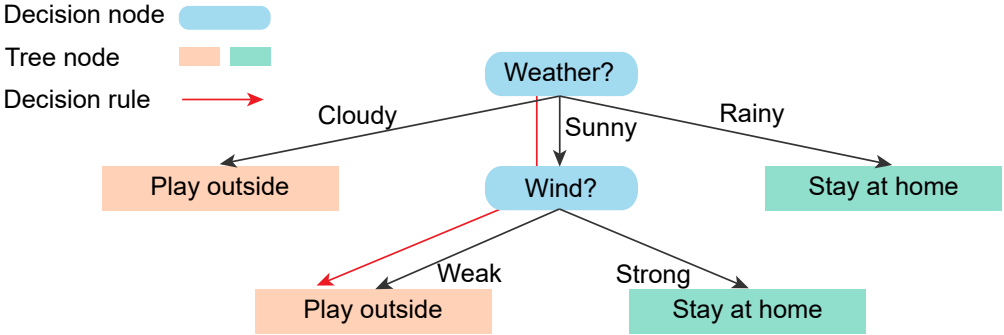
How to build a decision tree: big idea

1. Start with a dataset (e.g., features like age, income).
2. Pick a feature to split on (e.g., "Is age > 30?").
3. Keep splitting based on features until you reach a decision.

- Goal: Minimize uncertainty at each step (using metrics like entropy or Gini impurity).
  - $p(j|t)$: relevant frequency of class j at node t
  - Entropy: $\text{Entropy}(t) = -\sum_j p(j|t) \cdot \log p(j|t)$
  - Gini impurity: $\text{GINI}(t) = 1 - \sum_j [p(j|t)]^2$



Example of decision tree: minimize uncertainty at each step



- Begin with a table: 'Weather' is with less 'uncertainty' than 'Wind'

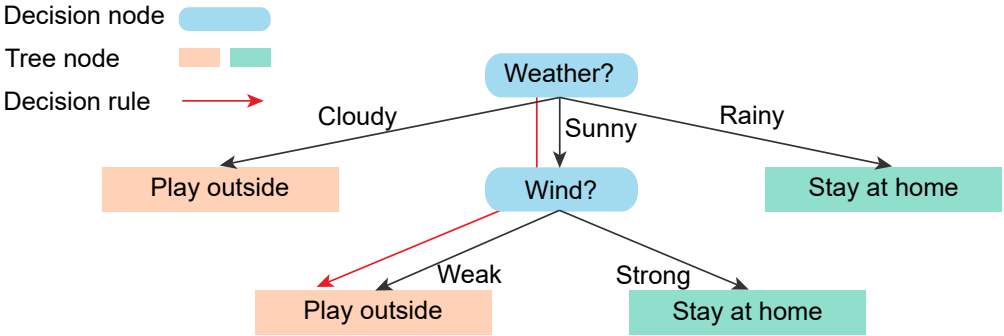
How to build a decision tree: details

Given a subset of data M (a node in a tree)

- **For each feature $h_i(x)$:**
  - Split data of M according to feature $h_i(x)$:
  - Compute classification error of split:
- **Chose feature $h^*(x)$ with lowest classification error (or other criteria)**
- **Stopping condition**
  - The tree doesn't grow forever—**stopping conditions** decide when to halt:
  - **Purity**: All data in a node belongs to one class (e.g., all "spam").
  - **Max Depth**: Limit how many levels the tree can have (e.g., stop at 5 splits).
  - **Min Samples**: Stop if too few data points remain (e.g., < 10 samples).
  - Prevents overfitting and keeps the tree manageable!



Example of decision tree: stop with a condition



- Begin with a table: 'Weather' is with less 'uncertainty' than 'Wind'
- Stop: Purity<0.5? Depth>3? # samples<3?



Pros & cons of decision tree

Pros

- Can handle large datasets
- Can handle mixed predictors (continuous, discrete, qualitative)
- Can ignore redundant variables
- Can easily handle missing data
- Easy to interpret if small

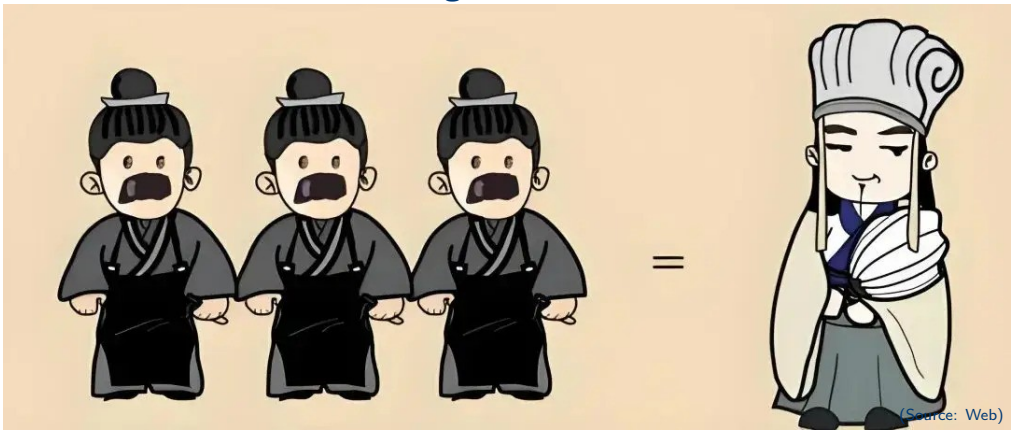
Cons

- Prediction performance is poor
- Does not generalize well
- Large trees are hard to interpret



How to make a decision tree stronger?

Ensemble learning: Wisdom of the crowds!



Ensemble learning: from one model to many

- Informal idea: combine several base learners to obtain a stronger model.
- Instead of trusting one model, we let multiple base learners vote or average together.
- **Key goal:** each base learner should be reasonably accurate, and the learners should not make exactly the same mistakes.
- We use tree ensemble as examples here (why?)
  - A single decision tree often produces unstable predictions.
  - A small change in the data may lead to a different split, and therefore a different tree.
  - Decision trees are super fast, flexible, and easy to train.



When does an ensemble help?

Consider a majority voting case:

- x_i: 1 sample (3 samples here)
- h_i: 1 base learner (3 base learners here)
- f: decision by majority voting

	x_1	x_2	x_3
h_1	✔	✔	✘
h_2	✔	✘	✔
h_3	✘	✔	✔
f	✔	✔	✔

Positive Effect

	x_1	x_2	x_3
h_1	✔	✔	✘
h_2	✔	✔	✘
h_3	✔	✔	✘
f	✔	✔	✘

No Effect

	x_1	x_2	x_3
h_1	✔	✘	✘
h_2	✘	✔	✘
h_3	✘	✘	✔
f	✘	✘	✘

Negative Effect

The base learners should have **high**

- Accuracy
- Diversity



Exercise: error rate of a majority voting ensemble

Consider a binary classification task. Assume that we have 3 **independent** base learners, and the error rate of each base learner is 0.2. What is the accuracy of their majority voting ensemble?

A 0.512

B 0.640

C 0.672

D 0.8

E 0.896

F 0.992



Error bound of majority voting

Consider a binary classification task. Assume that we have N **independent** base learners, and the error rate of each base learner is ε ($\varepsilon < 0.5$). Then the error rate of the ensemble is:

$$\sum_{k=0}^{\lfloor N/2 \rfloor} \binom{N}{k} (1-\varepsilon)^k \varepsilon^{N-k} \leq \exp\left(-\frac{1}{2} N(1-2\varepsilon)^2\right)$$

⊙ **Key takeaway:**

- When $\varepsilon < 0.5$, the error rate can **drop exponentially** as N increases.
- It works when each base learner beats random guessing
- Independence matters, because highly correlated learners will not give the same gain.
- This result explains why base learner needs accuracy



Proof sketch (Optional)

Theorem 1 (Hoeffding's Inequality)

Let $X_1, \ldots, X_N$ be independent bounded random variables with $X_i \in [a, b]$ for all i, where $-\infty < a \leq b < \infty$. Then for $\delta > 0$ we have

$$P\left(\sum_{i=1}^{N} X_i - \sum_{i=1}^{N} \mathbb{E}[X_i] \leq -\delta\right) \leq \exp\left(-\frac{2\delta^2}{N(b-a)^2}\right)$$

and

$$P\left(\sum_{i=1}^{N} X_i - \sum_{i=1}^{N} \mathbb{E}[X_i] \geq \delta\right) \leq \exp\left(-\frac{2\delta^2}{N(b-a)^2}\right)$$

- In our case, $X = X_1 + X_2 + \cdots + X_N$ is the number of base learners that make the correct predictions.
- $X_i \sim B(1, 1-\epsilon)$, $\mathbb{E}[X_i] = 1 - \varepsilon$, $a = 0$, $b = 1$.
- Hoeffding: $P(X - N(1-\varepsilon) \leq -\delta) \leq \exp(-2\delta^2/N)$



Proof (Optional)

- The error rate of the ensemble model is $P(X \leq N/2)$:
- $P(X \leq N/2) = P(X - N(1-\varepsilon) \leq N/2 - N(1-\varepsilon)) = P(X - N(1-\varepsilon) \leq -N(1/2-\varepsilon))$
- $P(X - N(1-\varepsilon) \leq -N(1/2-\varepsilon)) \leq \exp\left(-2N^2(1/2-\epsilon)^2/N\right) = \exp\left(-\frac{1}{2}N(1-2\varepsilon)^2\right)$
- Note: It requires $\delta = N(1/2-\varepsilon) > 0 \Rightarrow \varepsilon < 1/2$.
  - When $\varepsilon < 0.5$, the ensemble becomes more accurate as we add more independent base learners.
  - When $\varepsilon > 0.5$, the ensemble becomes worse as we add more independent base learners.
  - Exercise (optional): show the second point by yourself!

  $\Rightarrow$ Ensemble learning only succeeds when the base learners are at least better than random guessing.
  - How do we obtain a set of accurate and diverse base learners?

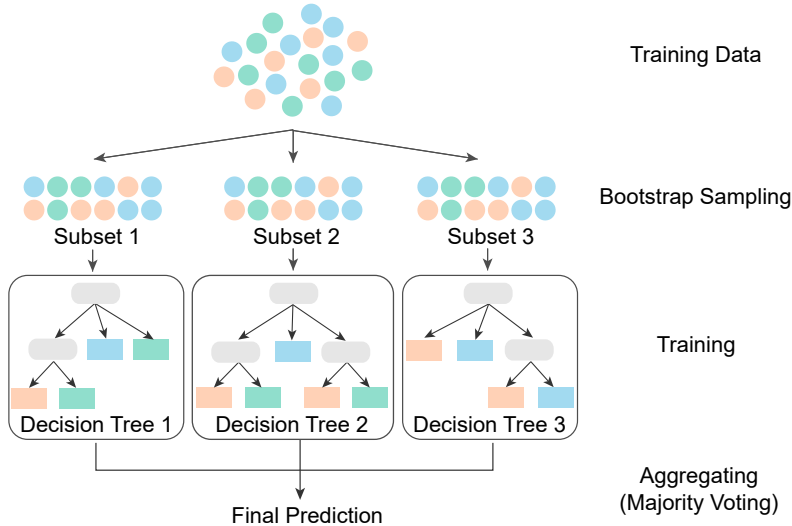


Bagging

- We want to build a set of high-performing and diverse base learners.
- Question: How to obtain a set of diverse base learners?
  - Use different models
  - Use different hyper-parameters
  - Use different training algorithms
  - Use different subsets of training data $\Rightarrow$ **Bagging**



Bagging


Training Data
Bootstrap Sampling
Subset 1
Subset 2
Subset 3
Training
Decision Tree 1
Decision Tree 2
Decision Tree 3
Aggregating
(Majority Voting)
Final Prediction

Bagging (Breiman, 1996)

- Bagging: Bootstrap AGGregatING
- Bootstrap sampling: selecting samples/objects with replacement, allowing the same object to be chosen multiple times
  - to get different subsets of the data
- Bootstrap aggregating: combine the model output
  - to ensure lower variance



How bagging works: step 1 - sampling

- Take your original dataset (e.g., 1000 data points).
- Generate N bootstrap samples (e.g., $N = 10$).
- Each set is the same size as the original but slightly different (by sampling **with replacement**)

- **Q:** Why not split data into K partitions?

  A: Waste of samples
- **Q:** Around 63.2% samples will be sampled/selected (why?)

  A: $\lim_{n\to\infty}(1 - 1/n)^n = 1/e \approx 0.368$



How bagging works: step 2 - training

- Train one model (e.g., decision tree) on each bootstrap sample.
- Each model learns slightly different patterns due to sample variation.

$\Rightarrow$ Result: N base learners (e.g., 10 decision trees).



How bagging works: step 3 - aggregation

- For **classification**: Use majority voting (e.g., 6 trees say "yes," 4 say "no" $\rightarrow$ "yes").
- For **regression**: Average the predictions.
- Combines strengths, reduces individual errors.



How many samples in a bootstrap: Demo

```python
import numpy as np
N = 10
idx_list = np.arange(N)
np.random.seed(42)
np.random.choice(idx_list, 10, replace=True)
# array([6, 3, 7, 4, 6, 9, 2, 6, 7, 4])

N = 100000
print(len(set(np.random.choice(np.arange(N), N, replace=True))) / N)
# 0.6322
```



Benefits of bagging

- Easy to implement
- Linear training/inference cost
- Easy to parallelize
- Model agnostic

- **Out-of-bag samples**



Out-of-bag samples

- Each bootstrap sample contains about 63.2% unique training examples.
- The remaining examples are called out-of-bag (OOB) samples.

Q: How can we use these out-of-bag samples?

A:

- Estimate prediction error without creating a separate validation split (OOB error)
- Hyper-parameter tuning
- Early-stopping



Out-of-bag estimate/OOB error

Procedure:

1. For each training example x_i, find the trees that did **not** use x_i during training.
2. Use only those trees to predict x_i.
3. Compare the prediction with the true label, and average over all training examples.

Implications:

- OOB error is a natural validation estimate for bagging and random forest.
- It plays a role similar to cross-validation, but it comes for free from bootstrap sampling.



Recap: cross-validation

1. Pick a bunch of interesting model classes (e.g., different ϕs: $\phi_1, \phi_2, \ldots, \phi_m$)
2. For each possible split of data into training set $\mathcal{D}$ and validation set $\mathcal{D}'$:

$$(\mathcal{D}_1, \mathcal{D}'_1), (\mathcal{D}_2, \mathcal{D}'_2), \ldots, (\mathcal{D}_K, \mathcal{D}'_K)$$

- Fit ('train') each model (class) ϕ on training set $\mathcal{D}$ and obtain hypothesis $h_{\phi,\mathcal{D}}$
- Compute error of each hypothesis on validation set $\mathcal{D}'$: $E_{\mathcal{D}'}(h_{\phi,\mathcal{D}})$
- Now, the validation set $\mathcal{D}'$ is not a single set
  - $\Rightarrow$ multiple validation sets $\mathcal{D}'$s

3. Estimate error of model as average of error on each $\mathcal{D}'$

$$\phi^* = \arg\min_{\phi_i} \frac{1}{K} \sum_{j=1}^{K} E_{\mathcal{D}'_j}(h_{\phi_i,\mathcal{D}_j})$$

4. Fit the model ϕ^* using the entire dataset.



Bagging helps variance reduction in prediction

- Bagging reduces variance by averaging predictions from many bootstrap-trained trees.
- Multiple bootstrap samples create multiple versions of the training set.
- Averaging these predictions makes the final model more stable than one single tree.



Correlated learners cannot effectively reduce variance

- Averaging i.i.d. variables scales their variance σ^2 to σ^2/B

$$\mathrm{Var}(\bar{X}) = \mathrm{Var}\left(\frac{1}{B}\sum_{i=1}^{B} X_i\right) = \frac{1}{B^2}\mathrm{Var}\left(\sum_{i=1}^{B} X_i\right) = \frac{1}{B^2} B\sigma^2$$

- Averaging i.d. variables with pairwise correlations ρ and variance σ^2 scales their final variance to be $(\rho + (1-\rho)/B)\,\sigma^2$

$$\mathrm{Var}\left(\sum_{i=1}^{B} X_i\right) = \sum_{i=1}^{B}\mathrm{Var}(X_i) + \sum_{i\neq j}\mathrm{Cov}(X_i, X_j) = B\sigma^2 + B(B-1)\rho\sigma^2$$

$$\rho_{X,Y} = \mathrm{corr}(X, Y) = \frac{\mathrm{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\mathrm{E}\left[(X - \mu_X)(Y - \mu_Y)\right]}{\sigma_X \sigma_Y}$$

$$\rho_{X,Y} = \frac{\mathrm{E}(XY) - \mathrm{E}(X)\mathrm{E}(Y)}{\sqrt{\mathrm{E}(X^2) - \mathrm{E}(X)^2} \cdot \sqrt{\mathrm{E}(Y^2) - \mathrm{E}(Y)^2}}$$



When $\rho = 0$. the variance is σ^2/B

Bagging: bias-variance decomposition

- Bagging reduces the variance while retaining the bias!

$$\mathbb{E}\left[\left(g-\hat{f}\right)^2\right]=\mathbb{E}\left[\left(g-\mathbb{E}\hat{f}+\mathbb{E}\hat{f}-\hat{f}\right)^2\right]=\underbrace{\left(g-\mathbb{E}\hat{f}\right)^2}_{\text{bias}}+\underbrace{\mathbb{E}\left[\left(\hat{f}-\mathbb{E}\hat{f}\right)^2\right]}_{\text{variance}}$$

  - g is the target function
  - $\hat{f}$ is the learned function
- **Bias** of bagged trees is the same as that of individual trees
- Reduction of variance depends on correlation among base learners.
  - If the trees are less correlated, averaging helps much more.

$\Rightarrow$ Bagging still leaves some correlation among trees, which motivates random forest.



Decision trees in bagging

- Pruning trees in a bagging model is usually not needed. Why?
- Decision trees are unstable base learners, so different bootstrap samples may produce very different trees.
- We want each tree to keep low bias, even if the individual tree has high variance.
- Unpruned trees are often more different from one another than pruned trees.

$\Rightarrow$ **How to "Decorrelate" the trees?**

A: **Random forest!**



Random forest

- Bagging adds diversity through bootstrap sampling.
- How to further **decorrelate** the trees generated for bagging?
- Ideas:
  - We can restrict how many times a feature can be used
  - We only allow a certain number of features
  - etc..



Random forest (Leo Breiman, 2001)

- As in bagging, choose B bootstrapped datasets.
- For each split in the B trees, consider only k features from the full feature set of size m.
- Different trees therefore explore different split candidates.
  - $k = m$: same as bagging.
  - $k < m$: random forest.

- Like a "forest" of more diverse trees working together.
- As in bagging, OOB samples provide a natural validation estimate during training.



Tuning random forest

- Random forest has multiple hyper-parameters to tune.
  - Level of each base learner:
    number of features to randomly select at each split
    minimum number of samples required at a leaf node
  - Level of the whole ensemble:
    total number of trees in the ensemble
    .....
- These parameters can be tuned by OOB error.



Random forest: pros & cons

- **Pros:**
  - Works well in practice, and k can be tuned as a hyper-parameter.
  - Introduces diversity through
    1) randomness in bootstrap sampling, and 2) randomness in feature selection.
  - Faster training than bagging (why?)

    A: Less features in base trees

- **Cons:**
  - If the number of relevant features is very small, a random subset may miss them.
  - Therefore, performance depends on a sensible choice of k.



Motivation for boosting

Q: Could we address the shortcomings of single decision tree models in some other way?

- **Idea:** students learn from their mistakes in multiple exams.

$\Rightarrow$ **Idea:** later models learn from the mistakes of earlier models.

A: Base learners are trained sequentially, and each learner uses information from the current ensemble errors.

Bagging reduces **variance** by averaging.

Boosting reduces **bias** by correcting errors step by step.



Boosting models

- **Core idea:** add simple base learners one by one into a stronger model.
- Each learner may be weak on its own, but a linear combination of them can be expressive.

$$T = \sum_h \lambda_h T_h$$

Q: Which learner should be added next, and how do we decide the weights λ_h?

A: Two representative methods:

- Gradient Boosting
- Adaptive Boosting (AdaBoost)



Gradient boosting

Core Idea

Build new models to gradually correct the mistakes of the current ensemble

Key features:

- A more general boosting method using gradient descent
- Works with any differentiable loss function
- Each new model is trained to correct the mistakes of the current ensemble.
  - In regression with squared-error loss, each new model fits the residuals
  - More generally, each new model approximates the negative gradient of the loss
- Powers tools like XGBoost, LightGBM: top performers in ML competitions!



Gradient boosting for squared-error regression

1. Fit a simple model $T = T^{(0)}$ on the training data $\{(x_1, y_1), \ldots, (x_N, y_N)\}$
2. Compute the residuals ('errors') $\{r_1, \ldots, r_N\}$ for model T
3. Fit a simple model $T^{(1)}$ to the current residuals, i.e., train using $\{(x_1, r_1), \ldots, (x_N, r_N)\}$
4. Build a new model $T \leftarrow T + \lambda T^{(1)}$
5. Compute the updated residuals $r_n \leftarrow r_n - \lambda T^{(1)}(x_n)$
6. Repeat steps 2-5 until the stopping condition met

⊙ **Question:** Why does fitting residuals help reduce the loss?



Recap: gradient descent

- We aim to minimize a objective function $f : \mathbb{R}^n \to R$
  - If the partial derivatives are simple,
    we can analytically find a stationary point where all the partial derivatives vanish.
  - If the objective function is convex,
    the stationary point is precisely the minimizer.



Recap: gradient descent

- In practice, the objective functions are complicated, and analytically finding the stationary point is intractable
- Instead, we use an iterative method called gradient descent

1. Initialize the variables at any value $x = [x_1, \ldots, x_n]$
2. Calculate the gradient at current value $\nabla f(x) = \left[\frac{\partial f(x)}{\partial x_1}, \ldots, \frac{\partial f(x)}{\partial x_n}\right]$
3. Adjust the variable values by some negative multiple of the gradient $x \leftarrow x - \lambda \nabla f(x)$



Gradient boosting as gradient descent: what is residual r_i

Take a regression task as an example

- Our objective function is the MSE:

$$\mathcal{L}_{\text{MSE}} = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

- Take gradient with respect to the predictions $\hat{y}$:
  $\nabla \mathcal{L}_{\text{MSE}} = -2[y_1 - \hat{y}_1, \ldots, y_n - \hat{y}_n]^\top = -2[r_1, \ldots, r_n]^\top$
- So, under MSE loss, fitting the negative gradient is equivalent to fitting the residuals
- Update the variable $\hat{y}_i \leftarrow \hat{y}_i + \lambda r_i$



Why not update the predictions directly?

- If we directly update each $\hat{y}_i$, we only obtain a better set of fitted values on the training samples
- But our goal is to learn a prediction function $F(x)$, not just a list of numbers

$$(\hat{y}_1, \ldots, \hat{y}_N)$$

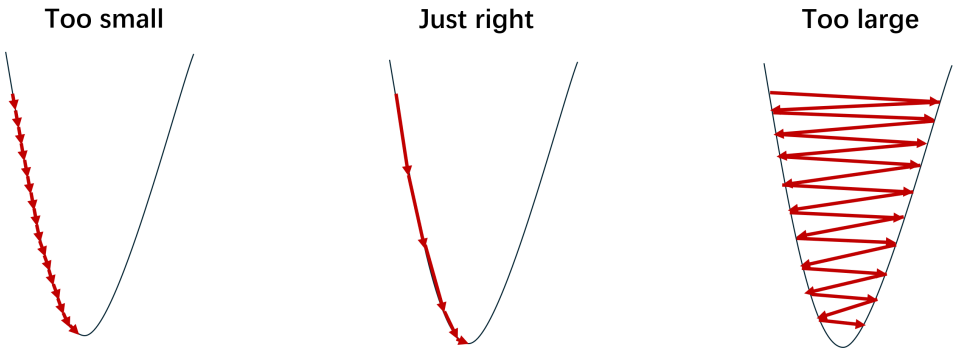
- This function F should work well with unseen x
- Therefore, instead of updating each prediction separately, we fit a simple model $h(x)$ to approximate the desired update
- The update becomes

$$F(x) \leftarrow F(x) + \lambda h(x)$$

- In practice, $h(x)$ is usually a small regression tree



How to choose the learning rate λ



How to choose the learning rate λ

- If λ is a constant, then it should be tuned through cross-validation

- It is better to use a variable λ
- Usually, λ should be related to the magnitude of the gradient
  - Around the optimum, when the gradient is small, λ should be small
  - Far from the optimum, when the gradient is large, λ should be large

  - Recap: variants of GD algorithms, e.g., Adagrad (Adaptive Gradient Algorithm)

$$[\mathbf{w}_{k+1}]_i \leftarrow [\mathbf{w}_k]_i - \frac{\eta}{M_{ii,k} + \varepsilon}[\nabla_{\mathbf{w}} C(\mathbf{w}_k)]_i \ , \quad \text{where } M_{ii,k} = \sum_{s=0}^{k} [\nabla_{\mathbf{w}} C(\mathbf{w}_s)]_i^2$$



Tuning gradient boosting

Tunable parameters:

- **Number of trees (B)**: Boosting can overfit unlike Bagging / RFs. Use cross-validation!
- **Shrinkage parameter (λ)**: small positive number that sets the learning rate
- **Number of splits in each tree (d)**: Usually just choose $d = 1$, i.e., tree stumps work well



Gradient boosting

Variants:

- Varying the tree constraints
- Weighting each tree to the additive sum using a learning rate (shrinkage)
- Sampling strategies: stochastic gradient boosting
- Regularization: ℓ_1-norm / ℓ_2-norm

- Successful trial (go exploring!): XGBoost, LightGBM ...



AdaBoost as a special case of boosting

- Historically, AdaBoost was not originally derived from gradient boosting.
- However, it can be interpreted after the fact as a special case of gradient boosting with a particular loss function.
- AdaBoost is equivalent to Gradient Boosting with the exponential loss for binary classification
- So AdaBoost can be viewed as
  - a boosting algorithm based on sample reweighting, and also
  - a special case of gradient boosting .



From gradient boosting to AdaBoost

- Gradient boosting applies whenever we optimize a differentiable loss.
- In classification, the direct 0-1 loss is not differentiable:

$$\mathcal{L}_{\text{0-1}} = \frac{1}{N} \sum_{n=1}^{N} \mathbb{1}(y_n \neq \hat{y}_n)$$

- So, instead of optimizing classification error directly, we often use a differentiable surrogate loss.
- AdaBoost can be interpreted in this way: it is closely related to boosting under the exponential loss.
- This viewpoint explains why AdaBoost leads naturally to sample reweighting.



Motivation for AdaBoost

- **Idea:** replace classification error with a differentiable loss that still penalizes wrong predictions heavily.
- A natural choice is the exponential loss:

$$\mathcal{L}_{\text{exp}} = \frac{1}{N}\sum_{n=1}^{N} \exp(-y_n F(x_n)), \qquad y_n \in \{-1, 1\}$$

- Why is this useful?
  - It is differentiable.
  - It penalizes misclassified points more heavily.
  - It leads naturally to sample reweighting.



Motivation for AdaBoost

- What does the gradient of exponential loss look like?

$$\nabla \mathcal{L}_{\text{exp}} = [-y_1 \exp(-y_1 F(x_1)), \ldots, -y_n \exp(-y_n F(x_n))]$$

- Each component can be written as $y_i \exp(-y_i F(x_i)) := w_i y_i$.
- Define the sample weight $w_i = \exp(-y_i F(x_i))$
- The gradient can be interpreted as a reweighting of the training samples.
  - When $F(x) = y$, the weight is small.
  - When $F(x) \neq y$, the weight is large.



AdaBoost: adaptive boosting

Core Idea

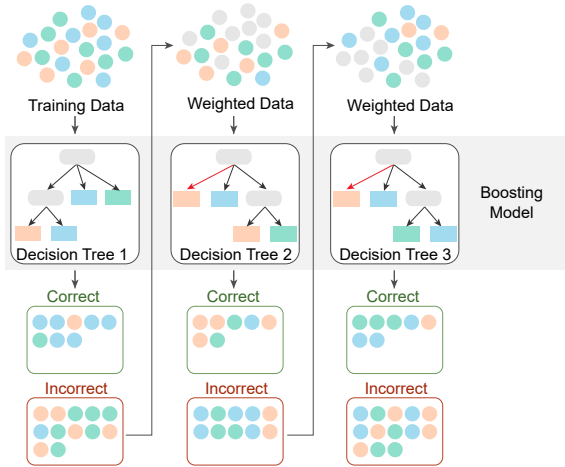
Adaptively reweights samples

Key Features:

- Put more weight on difficult to classify samples and less on those already handled well
- New base learners are added sequentially and focus on the more difficult patterns



AdaBoost


Training Data
Weighted Data
Weighted Data
Decision Tree 1
Decision Tree 2
Decision Tree 3
Boosting Model
Correct
Correct
Correct
Incorrect
Incorrect
Incorrect

AdaBoost: procedure

1. Initialize equal weights $w_i^{(1)} = 1/N$ for all training samples
2. For each iteration t in $[1, 2, \ldots,]$:

(a) Train a base learner $T^{(t)}$ on the weighted training data $\{(w_1^{(t)}, x_1, y_1), \ldots, (w_N^{(t)}, x_N, y_N)\}$ and compute the weighted error rate $\varepsilon^{(t)}$.

(b) If $\varepsilon^{(t)} > 0.5$, then stop.

(c) Compute the learner weight $\lambda^{(t)} = \frac{1}{2} \ln\left(\frac{1-\varepsilon^{(t)}}{\varepsilon^{(t)}}\right)$.

(d) Add the learner to the ensemble $F^{(t)} \leftarrow F^{(t-1)} + \lambda^{(t)} T^{(t)}$.

(e) Update sample weights:

$$w_i^{(t+1)} \leftarrow w_i^{(t)} \exp(-\lambda^{(t)} y_i T^{(t)}(x_i))/Z$$

3. The ensemble model will be the weighted combination of all base learners: $F = \sum_t \lambda^{(t)} T^{(t)}$.



Q: Why does this weighting make the next learner focus on previous mistakes?

Compute the learner weight

- $\lambda^{(t)}$ should minimize exponential loss of the ensemble model $F^{(t-1)} + \lambda T^{(t)}$

$$
\begin{aligned}
\mathcal{L}(\lambda) &= \sum_{i=1}^{N} \exp\left(-y_i \left(F^{(t-1)}(x_i) + \lambda T^{(t)}(x_i)\right)\right) \\
&= \sum_{i=1}^{N} \exp\left(-y_i F^{(t-1)}(x_i)\right) \exp\left(-y_i \lambda T^{(t)}(x_i)\right) \\
&\propto \sum_{i=1}^{N} w_i^{(t)} \exp\left(-y_i \lambda T^{(t)}(x_i)\right) \\
&= \sum_{i: y_i = T^{(t)}(x_i)} w_i^{(t)} e^{-\lambda} + \sum_{i: y_i \neq T^{(t)}(x_i)} w_i^{(t)} e^{\lambda}
\end{aligned}
$$



Compute the learner weight

$$\frac{\partial \mathcal{L}(\lambda)}{\partial \lambda} = -\sum_{i:y_i = T^{(t)}(x_i)} w_i^{(t)} e^{-\lambda} + \sum_{i:y_i \neq T^{(t)}(x_i)} w_i^{(t)} e^{\lambda} = 0$$

$$\sum_{i:y_i \neq T^{(t)}(x_i)} w_i^{(t)} e^{2\lambda} = \sum_{i:y_i = T^{(t)}(x_i)} w_i^{(t)}$$

$$\lambda = \frac{1}{2} \log \left(\frac{\sum_{i:y_i = T^{(t)}(x_i)} w_i^{(t)}}{\sum_{i:y_i \neq T^{(t)}(x_i)} w_i^{(t)}} \right) = \frac{1}{2} \log \left(\frac{1 - \varepsilon^{(t)}}{\varepsilon^{(t)}} \right)$$

- where $\varepsilon^{(t)}$ is the weighted error rate of the learner $T^{(t)}$



Update the sample weight

- $T^{(t+1)}$ should correct the errors in $F = F^{(t)} = \sum_{s=1}^{t} \lambda^{(s)} T^{(s)}$

Q: How does this simple update formula achieve this?

$$w_i^{(t+1)} \leftarrow w_i^{(t)} \exp(-y_i \lambda^{(t)} T^{(t)}(x_i))/Z$$

- Or $w_i^{(t+1)} \propto \exp(-y_i(\lambda^{(t)} T^{(t)}(x_i) + \cdots + \lambda^{(1)} T^{(1)}(x_i))) = \exp(-y_i F^{(t)}(x_i))$
- Obtain $T^{(t+1)}$ that minimize exponential loss for $F + T^{(t+1)}$

$$\begin{aligned}
&\sum_{i=1}^{N} \left[\exp\left(-y_i(F^{(t)}(x_i) + T^{(t+1)}(x_i))\right)\right] \\
&= \sum_{i=1}^{N} \left[\exp(-y_i F^{(t)}(x_i)) \exp(-y_i T^{(t+1)}(x_i))\right] \\
&\approx \sum_{i=1}^{N} \left[\exp(-y_i F^{(t)}(x_i))(1 - y_i T^{(t+1)}(x_i) + \frac{1}{2} y_i^2 T^{(t+1)}(x_i)^2)\right] \text{ (Taylor expansion)} \\
&= \sum_{i=1}^{N} \left[\exp(-y_i F^{(t)}(x_i))(\frac{3}{2} - y_i T^{(t+1)}(x_i))\right] \text{ (Binary classification: } y_i, T^{(t+1)}(x_i) = \pm 1\text{)}
\end{aligned}$$



Update the sample weight

- Choice of $T^{(t+1)}$ CANNOT change $\left[\exp(-y_i F^{(t)}(x_i)) \cdot 3/2\right]$.

$$
\begin{aligned}
T^{(t+1)*} &= \arg\max_{T^{(t+1)}} \sum_{i=1}^{N} \left[\exp(-y_i F^{(t)}(x_i)) y_i T^{(t+1)}(x_i)\right] \\
&= \arg\max_{T^{(t+1)}} \sum_{i=1}^{N} \left[\frac{\exp(-y_i F^{(t)}(x_i))}{\sum_{i=1}^{N} \exp(-y_i F^{(t)}(x_i))} y_i T^{(t+1)}(x_i)\right] \\
&= \arg\max_{T^{(t+1)}} \sum_{i=1}^{N} \left[w_i^{(t+1)} y_i T^{(t+1)}(x_i)\right]
\end{aligned}
$$

$\Leftrightarrow$ Minimize the **weighted** error rate

A: That's why we trained $T^{(t+1)}$ using the weights $w_i^{(t+1)} \propto \exp(-y_i F^{(t)}(x_i))$.

- How to update efficiently:

$$w_i^{(t+1)} / w_i^{(t)} \propto \exp\left(-y_i (F^{(t)}(x_i) - F^{(t-1)}(x_i))\right) = \exp\left(-y_i \lambda^{(t)} T^{(t)}(x_i)\right)$$



Advantages of boosting methods

- **High Accuracy:**
  - AdaBoost: Turns weak base learners into a strong classifier (e.g., spam detection).
  - Gradient Boosting: Often beats other methods (e.g., powers XGBoost wins in competitions).

- **Flexibility:**
  - AdaBoost: Simple and fast for binary tasks.
  - Gradient Boosting: Handles any (differentiable) loss function – regression, classification, ranking.

- Bonus: Both can rank feature importance (Gradient Boosting especially).



Limitations of boosting methods

- **Overfitting Risk:**
  - AdaBoost: Sensitive to noisy data and outliers (overemphasizes them).
  - Gradient Boosting: Too many trees or poor tuning can overfit.

- **Computational Cost:**
  - Both: Sequential training—slower than parallel methods like bagging.
  - Gradient Boosting: Especially resource-heavy with many iterations.

- Tuning needed (e.g., learning rate, rounds) for best results.



Practical tips for boosting methods

- Use simple base learners (e.g., stumps) for best results.
  - Decision stump: a type of decision tree that consists of a single decision node and two leaf nodes
- Limit rounds (e.g., 50-100) to avoid overfitting.
- Preprocess data to reduce noise and outliers.



Advanced tree ensembles: XGBoost and LightGBM

- **XGBoost** is a powerful tree ensemble method with regularization and strong engineering support.
- **LightGBM** is designed for faster training and lower memory usage, especially on large datasets.
- In practice, XGBoost is a strong default choice, while LightGBM is attractive when efficiency is especially important.



Comparison: four ensemble methods

- **Bagging**: parallel training; bootstrap sampling plus averaging; strong at variance reduction; weaker when trees are still highly correlated.
- **Random forest**: parallel training; bagging plus random feature subsets; stronger diversity than bagging; may miss important features when k is too small.
- **Gradient boosting**: sequential training; fits residuals or negative gradients; often gives very high accuracy; requires careful tuning.
- **AdaBoost**: sequential training; reweights difficult samples; simple and intuitive for classification; sensitive to noise and outliers.



Summary: ensemble learning

- Ensemble learning combines multiple base learners into a stronger predictor.
- Bagging reduces variance by averaging models trained on bootstrap samples.
- Random forest further reduces correlation by randomizing the feature subset at each split.
- Boosting builds learners sequentially so that later learners correct earlier mistakes.
- In practice, bagging and random forest are robust defaults, while boosting often gives the best accuracy after tuning.



Thanks for listening

Tell us your question/feedback via QR code/Email/Teams.

⊙ Slides credit: some slides are adapted from (alphabetical order)
Pavlos Protopapas, Kevin Rader and Chris Tanner (Harvard), Ryan Tibshirani (CMU);
Dr. Zixin Zhong contributed many valuable corrections and additions.

⊙ Book: 'Pattern Recognition and Machine Learning' by Christopher M. Bishop

'The Elements of Statistical Learning' by Trevor Hastie, Robert Tibshirani, Jerome Friedman

